CALLWISE

Interview walkthrough

Real-Time Call Objection Handling Assistant

IMPORTANT This is a simulated prototype. It does not place calls, capture audio, transcribe speech, or invoke Twilio, LiveKit, or OpenAI.

30-second introduction

Callwise is a browser-based prototype for helping sales and support agents respond thoughtfully during customer calls. It demonstrates a call cockpit with a transcript experience, objection cues, suggested and alternative replies, sentiment, and a post-call summary. The demo is deliberately local and scripted so the experience can be explored without provider accounts, phone numbers, or API keys.

Problem and user journey

1. Choose a budget or existing-provider example and start a simulated call.
2. Sample dialogue appears. A scripted objection reveals a suggested response, alternatives, and an illustrative sentiment reading.
3. Select a suggestion to copy it, adapt it in your own words, and end the call.
4. Review the sample session summary and illustrative team insights.

A guided simulation makes the key product states repeatable in an interview. It is not a live telephony or AI demonstration.

Architecture

Browser UI Call cockpit, scripted state machine, transcript and coaching display	**Application backend** Future authenticated session and provider boundary
Twilio Future phone call connection and verified webhooks	**LiveKit + OpenAI** Future scoped audio room, bridging, transcription and analysis

In this repository, app.js owns local sample scenarios and the UI state machine. server.mjs serves the static app and reports all providers disabled. It does not process calls. For production, an authenticated backend would create sessions, keep secrets server-side, bridge authorized Twilio media into a scoped LiveKit room, and send consented audio to OpenAI.

DEMO & DISCUSSION

Walkthrough, trade-offs & next steps

A concise guide to explaining what works today and what production would require.

Run the demo

From the repository folder, run `node server.mjs` and open `http://127.0.0.1:4173`. Choose a scenario, start the demo call, watch transcript lines arrive, inspect the coaching cue, select a reply, end the call, and open Call insights. `GET /api/health` explicitly reports simulated mode and disabled providers.

Engineering decisions

- **No dependencies:** the demo runs on a clean Node.js setup without installs or credentials.
- **Simulation over fake integration:** sample dialogue reliably demonstrates the UX while clearly separating the idea from actual telephony, streaming, or model inference.
- **Provider boundary:** credentials and vendor calls belong on an authenticated backend, never in browser code.
- **Honest metrics:** aggregate charts are labeled illustrative; a sample comparison does not establish causal impact.
- **Privacy by default:** this demo captures and stores no call audio.

Production roadmap

- Define authentication, roles, participant consent, session authorization, and audit events.
- Build Twilio call-control and verified-webhook adapters; issue scoped LiveKit tokens and bridge media server-side; add OpenAI transcription and analysis through protected services.
- Add streaming events, reconnection/backpressure handling, observability, and clear provider failure states.
- Evaluate transcript and objection quality with human review; test latency, security, and accessibility.
- Set access, retention, deletion, and incident-response policies before handling real call data.

Responsible use

Before recording, transcribing, streaming, or analyzing real calls, obtain any consent required from every participant under applicable law and company policy. Provide clear notice and opt-out, minimize collected data, restrict access, set retention/deletion rules, and do not use AI sentiment or suggestions as the sole basis for consequential decisions.

Interview takeaway

The prototype proves the user journey, not the vendor integration. The production roadmap makes the boundary, privacy requirements, and validation work explicit.